Computer-Aided Design of Integrated Circuits: Sizing an Opamp with Reinforcement Learning & LLM-based Agentic Harness

Mohsen Ahmadzadeh
mohsen.ahmadzadeh@esat.kuleuven.be

Pierre Graindorge
pierre.graindorge@esat.kuleuven.be

November 3, 2026

Introduction

This project involves utilizing a Reinforcement Learning algorithm, specifically the Twin-Delayed Deep Deterministic Policy Gradient (TD3) agent, in an explainable LLM-based agentic harness (using Antigravity CLI) to determine optimal sizing for a two-stage Miller opamp. The project is structured into four key milestones.

1. Simulator Interface

In the first milestone, your task is to complete the circuit netlist to ensure it outputs all the necessary data for measuring specifications. You can find necessary information in the ngspice reference manual. Subsequently, you will complete the Python code for extracting the specifications, including Gain, Noise, Phase Margin, Unity GBW, Slew Rate, Estimated Area (already provided), and Total Current.

Additionally, you are required to write a unit test file to verify the accuracy of these measurements across different sets of sizes. To achieve this, you may use ngspice plots to define reference designs or leverage example references provided by the TAs during the sessions.

Relevant files

- circuit netlist: `TwoStage.cir`
- circuit yaml file: `TwoStage.yaml`
- simulator wrapper: `ngspice_wrapper.py`
- measurement file: `dut_testbench.py`

What to Report

Describe the circuit topology that is used in the project. Explain how you measure each performance metric (spec). Provide a proof of the correctness of your measurement by describing your unit test.

2. Circuit Environment

In the second milestone, you need to complete the Python code for the circuit environment. This environment has to encapsulate critical components, such as the definition and normalization/refinement of states and actions, reward computation, and the implementation of reset and step functions. The ngspice wrapper and measurement scripts will be used in this file, and you may also use the provided plotting functionality to update output plots within the step function.

To validate your implementation, develop a straightforward unit test file to assess the netlists and rewards generated for different designs (i.e., sets of sizes/actions). In your test, you can use manual calculations of normalized specs, rewards, etc. as reference values to make sure your code works well.

Relevant Files

- circuit environment file: `circuit_env.py`

What to report

Explain the details of how you refine the action to their real values and how you normalize the specs. Explain the functionality of step and reset in the environment. Show a proof of the correctness of your implementation by explaining your unit test file.

3. RL Agent

The third milestone involves completing the TD3 agent code based on the provided algorithm. The code for the Actor and Critic neural networks, as well as the replay memory, is already provided. Using these components, you will implement the agent's primary learning function.

Furthermore, you need to tune some required hyperparameters in a runner file that employs the circuit environment and the TD3 agent within an episodic main loop of steps. You then need to tune the required. Another RL agent will be provided by the TAs as a reference for comparison.

The required hyperparameters to be tuned are: noise sigma, batch size, warmup steps, learning rates of the actor and critic networks.

Relevant Files

- The TD3 agent files: `agent.py` and `buffer.py`
- The TD3 runner file: `td3_runner.py`
- The baseline agent file: `baseline_runner.py`

What to Report

First, explain your implementation of the TD3 agent. Then, describe your episodic loop in the runner file as well as your reasons for your choice of hyperparameter values.

4. LLM-based Agentic Harness

In the final phase, you first learn how to use Antigravity CLI, which also makes LLMs from other vendors freely available. Then, you should use that knowledge to build an LLM-based agentic harness using MCP tools and instruction skills that can reason about the project circuit, explain its functionality, refine the sizing parameters, provide feedback, and run the RL-based sizer as a tool based on its own judgment when necessary. Good prompt and context engineering is

needed. Use your own expertise and break the task of sizing the opamp into different sub-tasks and assign a separate LLM agent (with specialized prompts, MCP tools, and instruction skills) to each sub-task. Manage the context length and provide necessary context to different LLM agents from previous LLM outputs (memory). The harness should provide reasoning for the decision steps taken. The goal is to have a sizer that is no longer a black box! Expert designers need interpretability of the decisions and found solutions in order to be able to rely on the software and use it in practice.

Relevant Files

No relevant files are provided for this section.

What to Report

Explain how you break the task and which LLM agents (with which prompts, context, and past memories) you use to get the best explainable sizing harness. Describe which MCP tools you use and how they work, how you manage context and memory, and which instruction skills you use and how they should work. Explain each agent and the full harness. Show examples of explainability. Provide the best found solutions along with reasoning. Try to achieve good solutions with the fewest number of simulations possible.

Finalize and Report

Upon completing all the milestones, you will have developed a functional RL-based sizing algorithm along with an LLM-based agentic harness that can use the RL-Sizer if necessary.

For the RL sizer part, you are expected to generate and report the two required output plots: 1) The Maximum FoM reached per simulation step, and 2) The Current-Area Pareto front of solutions. You are required to compare your results to the results achieved by the baseline agent within 5k simulations.

For the LLM-based agentic harness, you are expected to provide a comparison of speed (in terms of runtime) for the best found solution using the LLM-based agentic harness as opposed to purely using the RL sizer. This comparison can either be shown by a plot (runtime on the x axis and Maximum FoM reached on the y axis) or by a table that compares the runtime for solutions with similar FoMs using the 2 approaches. You are also expected to show examples of interpretability of your harness to prove that it provides traceable reasoning and decision steps.

You will prepare a project report, adhering to the **maximum length of 10 pages** as discussed in class.

Finally, you have to upload the report, your entire project directory, and the generated files of your Pareto solution designs as one single compressed archive (.zip, .tar.gz, .tar.xz, .7z...).

The deadline for the submission is **Sunday, November 8th, 23:59 Brussels Time!**

Contents of the Submission

A compressed archive named

`[student_number]_[last_name]_G[group_number].zip`

containing:

- Your report as a .pdf file.
- Your project directory including all the necessary files for running your algorithm.

- One folder named '`Solutions`' containing the netlists of your Pareto solutions (after 5k simulations) and one .csv file that lists the details of the parameters, specs, and the reward (FoM) of the solutions.

Good Luck!

Appendix: Optional Tasks

After you have completed the entire project, if you have time left, you can take up the challenge of completing one of the following optional tasks. It is important that the entire project be completed and functioning before you consider an optional task.

1. Modify your setup and algorithm in a way that it finds **PVT-Robust solutions**. In this case, solutions are only accepted when they are robust in 12 corners {TT, SS, FF} $\times$ {1.2V, 1.0V} $\times$ {0 °C, 100°C}.
2. Write the necessary code for running cheaper simulations or analyses (e.g., DC, AC, transient, or selected corners) and acquiring useful circuit or transistor data (Vgs, Vth, Vds, gm, Id, etc.). Then, expose them as MCP tools for LLMs and write the agents in your harness in a way that they can benefit from these cheaper simulations or analyses whenever beneficial/necessary.